

How BTS-MESH, CRUCIBLE, and COSMOS work

OS first: SCAR, ROLD, carry-over. MOTIF is how it builds.

Written-description appendices for US provisionals

Not a USPTO filing. Not legal advice. Attach these PDFs to the provisional packets at filing. Do not add them after a filing date. Keith + Legal file. This TUI does not click USPTO.

September 2026

Appendix - COSMOS is the OS (not MOTIF)

Kind: written description for Legal. Not a USPTO filing. **Keith 2026-09-07:** *The SCAR system, the ROLD, the CARRY OVER SYSTEM - COSMOS is much more than just MOTIF.*

MOTIF (P01) is the **method that builds** the house. COSMOS is the **house**: an operating system whose law is earned from measured failures, whose desk of record is ROLD, and whose sessions **carry over** as a structural close/open, not as chat memory.

Three organs MOTIF is not

1. The SCAR system

A **scar** is a failure that was **measured**, then encoded as law. ROLD SCARS.md (local v:\Ai\ROLD, GrokBot tree): append-only, dated, numbered, **never edited**. Wrong entries stay; a later correction names the entry it corrects. That is ADR/incident class (Nygard): context to what happened to consequence.

COSMOS turns scars into **kernel interfaces** (AD-11): workers cannot import around them. The worst class is **placation** (docs/SCAR_PLACATION.md, 2026-08-25): a plausible claim of compliance that the artifact contradicts. Guard: every consequential action emits a machine-checkable artifact; a report **quotes the artifact**, never the intention; missing or contradicting artifact is a refusal, surfaced first. Runtime-binding (P04) is this guard as a gate.

Empty-dir identity, two-writer deletion, green-log DONE, bash-mount CRLF phantoms - each is a scar that became a primitive (resolver, one pen, live emit, host-side ground truth). MOTIF *uses* those primitives. It did not invent them.

2. The ROLD

ROLD = the desk of law / repository of documents. Keith 2026-07-30: a repository, not a flat dump. Organising rule: **a document is defined by what makes it change**. If two things in one file change for different reasons, they are two files.

Always-granted across streams: rules, glossary, scars, rails, routines. **Ask the tools index first. Do not rebuild.** ROLD Rule 1: SGH and GEM first, both, in parallel, on DOM. The index is identity of capability; existence of a folder is not a tool.

ROLD is not MOTIF stage 2 RESEARCH. RESEARCH *reads* ROLD. The desk exists whether or not a MOTIF tick is running.

3. The carry-over system

Close of a session **must** write a signed context manifest (SEED.json, declared length/HMAC): inherited facts, active leases, open watchers, handoff recipient. Close without it is **OPEN_CONTEXT**. Start refuses NO_SEED / BAD_SEED / IDENTITY_MISMATCH (P11, AD-10).

BUCm.toml is the lightweight session pointer beside the SEED - one truth, never a competing second handoff.

Resume gate (all streams): after BootUP loads carry-over, one option (resume all / subset / hold). No affirmative selection to **AUTO-RESUME** on the native 15s clock. Default is MOTION. A **HOLD** never self-clears. Two pause kinds are not the same.

Carry-over is why COSMOS survives context death. MOTIF is what the clock drives **after** the seed is accepted.

How they sit together

Organ	Job	Packet
SCAR system	Law earned from measured failure; anti-placation	P04 + kernel AD-11

Organ	Job	Packet
ROLD	Desk of record; ask before rebuild	supporting disclosure (this appendix)
Carry-over	Signed close; resume gate; auto-resession	P11
Core	One writer, ledger, fence	P07, P10
Ingress	Voice/Chatbox to GitHub drop to daemon to agent to audit	P13
MOTIF	How the OS builds the next organ	P01 (DEFINE first)

File the OS. File MOTIF. Do not file MOTIF as if it were the OS.

Attach to

P04, P07, P10, P11, P13, and the how-it-works pack. Duplicate at filing.

Appendix - How COSMOS works

Kind: written description / drawings-in-prose for Legal to attach to US provisionals. **Status:** not a USPTO filing. Not legal advice. Not a novelty opinion. **Source of truth:** docs/FINAL_ARCHITECTURE.md ratified 2026-08-23 (docs/RATIFIED.md); live Core tree_id=KMesh-COSMOS-live on this machine. **Maps to packets:** P07 (ledger + fence), P10 (resolver), P11 (SEED), P08 (DOM-first), P09 (spend). MOTIF (P01) is how the OS builds the next organ - not the OS. See **Appendix - COSMOS is the OS** (SCAR, ROLD, carry-over).

One paragraph (the invention as an OS)

One resident Windows service - **COSMOS Core** - is the sole authority: API gateway, scheduler, lease arbiter with fencing tokens, spend gate, return-watcher, registry + prober, and the single writer of an **append-only, hash-chained, service-signed JSONL ledger** on a verified native volume. Everything else - queue views, registries, dashboards (KDash / cDeck), spend totals - is a **rebuildable projection** (service-private SQLite allowed as cache, never authority). Large artifacts live in a **content-addressed store** (filename = hash; the ledger holds the live pointer). Workers (native, DOM, cloud) run in attempt-private workspaces and publish only through a **fenced commit gateway** that demands a fencing token and expected input hashes. The service is a **modular monolith, split-ready**: module interfaces are RPC-shaped so any module can later become a process without breaking the one versioned external API that cDeck, voice, and other clients consume.

COSMOS is the **operating system**. Products (Forge, Crucible, Diligence, Docket, UPS, Differentiator) are **skins** on that OS, not second Cores.

What a later claim must be able to point at

A person of ordinary skill, given this appendix plus the live tree, can build:

- 1 A single process that is API + scheduler + arbiter + ledger writer.
- 2 A framed JSONL ledger with hash chain and service signature; replay rebuilds projections; a corrupt segment **REFUSES** rather than repairing in place.
- 3 A boot resolver that takes **one** install-configured root, verified by sentinel **content** - existence of an empty directory is not identity.
- 4 Workers that cannot write the live tree except through the fenced commit gateway.
- 5 One versioned HTTP API (/api/v1/. . .) as the only external surface.
- 6 DOM as a first-class rail (unmetered path) with API as fallback, both audited.
- 7 A spend gate that will not silently widen a cap.
- 8 A signed session-close manifest (SEED) so carry-over is structural.

Decisions (ratified 2026-08-23)

#	Decision
1	One resident service. Fail-closed. Never a second unsynchronized writer.
2	Leases + monotonic fencing tokens + fenced commit gateway. Advisory locking is dead.
3	Ledger = framed, hash-chained, signed JSONL. Corrupt segment ⇒ REFUSE + incident.
4	Queue = immutable manifests + ledger lifecycle events. SQLite is projection only.
5	Resolver instantiated at boot; service cannot go READY without sentinel-verified root.
6	DOM is a first-class scheduler rail (contained workers, typed failures).
7	One versioned API. UI may deploy separately; authority may not.
8	Backup: policy/verification in Core; execution as scheduled jobs; rehearse-restore is first-class.
9	Compatibility lane: legacy mutable-file tools SERIALIZED until they earn parallelism.
10	Context manifests at session close. Closure without a valid manifest ⇒ OPEN_CONTEXT.
11	Scar-derived primitives are kernel interfaces; workers cannot import around them.

Two roots (do not conflate)

- **Repo tree** - tracked code and governance (cosmos/, docs/, tests/).
- **Runtime root** - the running instance (live/: state, ledger, queue, registry, config).

Identified by `.cosmos-root.json` (system=COSMOS, tree_id=KMesh-COSMOS-live).

The resolver takes one configured root. No drive literal as identity, no parent-walking fallback ladder, no import-time side effects.

Method that builds the OS (MOTIF)

COSMOS builds itself on directional input. The loop is **nine** stages: **DEFINE** to RESEARCH to ARCH to CONSENSUS to BUILD to CRITICS to CONSENSUS to IMPROVE to ITERATE (back to **DEFINE**, then RESEARCH). Dual-lane BUILD: two builders, **no shared context**, different families. Critics judge *is this the thing we decided*, not house style. One disposer (CCr) writes the live tree. The runtime-binding gate is a value **only the live tree can emit** - never an exit code, never a green log.

What this is not

Not a second blockchain. Not git-as-authority. Not “event sourcing exists.” Not a dashboard that is itself the OS. Not OpenWork (orch harness) and not cDeck (skin). cDeck is a client of `/api/v1`. OpenWork is a separate app/window.

Already public (clock)

GitHub cosmos created 2026-08-23T06:42:12Z. Ratification record that day. Predecessor mesh named on GitHub `bts-mesh` 2026-08-16. July 2026 operation is **local BTS-MESH run records**, not that GitHub date. See appendix How BTS-MESH works.

Attach to

FILE packets P01, P04, P07, P08, P09, P10, P11 at minimum. Duplicate this PDF into each provisional so each filing is enabling on its own (a provisional cannot claim benefit of a sister provisional).

Appendix - How CRUCIBLE works

Kind: written description for Legal to attach to US provisionals. **Status:** not a USPTO filing. **P12 (role-triad as the invention) remains HOLD.** This appendix describes the **occupancy engine applied to a legal packet** (a skin of P05), not “AI plaintiff / defense / judge” as a claim. **In-tree:** POST /api/v1/crucible; 501 CRUCIBLE_NOT_RUNNABLE when no critic dispatchers are composed. Seats in model_rater: crucible.plaintiff, crucible.defense, crucible.judge. Product profile: docs/PROFILES.md. Federated embodiment: Grayson runs Crucible as the **app** on **his** Core (docs/federation/GRAYSON.md); host still NO_HOST until Keith names it.

What it is

Crucible is a **product** on the COSMOS OS. A case (or other legal packet) is evaluated by **named seats** that run **independently first**, then argue. One **disposer** holds the write path. Spend is gated. Critics are a **different family** from the builders and judge the *decision*, not style.

The same occupancy is reused as other skins by changing the **packet**, not by cloning the stack:

Skin	Packet	Seats (embodiments)
Forge	code / work order	CCr + N adversarial coders
Crucible	legal casefile	plaintiff / defense / judge as seats , not as the claimed invention
Diligence	data room / 10-K / deck	bull / bear / independent risk
Differentiator	anonymized clinical case	independent opinions, then argue
Docket	IP packet	applicant / examiner / prior-art

Roles are **embodiments**. The method is: isolate proposers, different-family critics, one disposer, runtime-bind, ledger the round.

How a round runs (enabling)

- 1 Operator (or federated peer) presents a packet and named seats. Each seat is a model id from the local catalog / occupancy file, not a hard-coded vendor.
- 2 **Independent first.** Each proposer receives the packet and the task. They do **not** share a transcript mid-pass. No “second agent based on the first.”
- 3 Artifacts land as proposals (files / job results), not as a merged chat.
- 4 **Argue.** A later stage may exchange the isolated first takes. Disagreement is the signal, not a defect.
- 5 **Critics** (different family) compare the round to the **decided** rubric.
- 6 **One disposer** accepts or refuses. Core ledgers the round. cDeck Review pane paints FINDINGS; it does not invent an approve/reject verb the OS lacks.
- 7 POST /api/v1/crucible without composed critics is **501** - an honest refusal, not a stub job. Fail-closed.

Federation (shape, not a live peer)

COSMOS is the OS. Crucible is the application a peer can run. Grayson: thin Core on **his** PC, **his** keys, **his** tree. Updates/comms from a hub Keith names later (T7 or SRV1 - both NO_HOST today). Mailbox + one versioned API + lease **before** any write that touches Keith’s install. Do not share v:\A. Do not invent a hostname.

cDeck parallel instances (Forge window + Crucible window) are two **clients** of one API, the same sit as Grok.com + OpenWork today. Not two Cores.

What not to claim

Do **not** claim “AI plaintiff, defense, and judge” as the invention. That triad is crowded: SimuCourt/AgentsCourt arXiv:2403.02959; AgentCourt arXiv:2408.08089; **CN119168059B granted** 2025-07-15; **US20260037351A1** claim 21; jury-side US20250148558A1. File Crucible only as **P05 occupancy applied to a legal packet**. Else

fold into P05 and spend the 12th slot on UPS-JUDGE with Keith's July pack.

Name collisions (not TESS): CrucibleTech RN 8043152; Star Lab CRUCIBLE RN 5023199; Crucible Discovery; Destiny PvP fame.

Already public (clock)

GitHub bts-mesh **About** text names "CRUCIBLE method" as of 2026-08-16T10:21:14Z. Tracked files in that repo: **zero** hits for CRUCIBLE. Public date is a **name**, not a method disclosure in code. US grace for that string is thin.

Attach to

P05 (parent occupancy) at minimum. P12 only if Legal files occupancy-not-roles. Do not attach this appendix as if it cured a PDJ claim.

Appendix - How BTS-MESH works (July 2026 predecessor)

Kind: written description / provenance for Legal to attach to US provisionals. **Status:** not a USPTO filing. Excerpts from the local predecessor tree `V:\Ai\BTS_MESH` (GrokBot pen). This TUI **read** those files; it does **not** write that tree. Do not commit keys, OAuth, or live ledgers. **Purpose:** prove the mesh **ran in early/mid July 2026**, and describe the architecture COSMOS later replaced - so provisionals are not a history rewrite.

Dates (measured)

Fact	Date	Where
North-star architecture written	2026-07-10	BTS_MESH_NORTH_STAR.md (local)
Working foundation README	2026-07-10 / README stamp 2026-07-11	README.md - bus, registry, cards, watcher, orchestrator; 29 tests
SOP / wiring / state feed	2026-07-14	BTS_SOP.md, WIRING.md, BTS_STATE_2026-07-14.md
GitHub bts-mesh created	2026-08-16T10:21:14Z	public repo; About names CRUCIBLE; tracked files have 0 CRUCIBLE hits
COSMOS architecture ratified	2026-08-23	GitHub cosmos; docs/RATIFIED.md

July operation = **local run records**. GitHub August 16 is **not** the start of operation. Private-now does not un-publish GitHub.

The goal (Keith, recorded 2026-07-10)

Grok could start a new session of another model (or ChatGPT) and load/point to needed context, and vice-versa, as new sessions and topics spawn - multi-session, multi-agent, multi-AI simultaneous coordination - and get Keith out of the middle as the human message-wire.

The one insight: no consumer chat tab can autonomously boot **and seed** another vendor's session. That capability has to live in a **local orchestrator** on the operator's machine.

Five components (as of 2026-07-10, working code)

- 1 **Shared bus** (`bts_bus.py`). Append-only, author-prefixed signal log. Payloads written atomically (temp + rename) so no reader sees a half-file. Per-reader cursor; self-filtering so a side never eats its own signals.
- 2 **Session seed**. Every spawned session starts by reading a `SESSION_SEED-<topic>.md` pointer. "Load the needed context" = read the seed. Orchestrator auto-creates seeds from a template. This is BootUP generalized.
- 3 **Session registry** (`SESSION_REGISTRY.json`). Single source of truth for {agent, topic, url, status, tasks}. Prevents two sessions doing the same work.
- 4 **Capability cards + routing**. Each AI advertises what it is best at; the daemon routes a new topic to a node. Cards are data (`cards/*.json`).
- 5 **Orchestrator daemon** (`bts_orchestrator.py`). Consumes the bus, updates the registry, handles `SPAWN_SESSION` (route to seed to register to queue launch). `launch_session()` was the remaining stub (Chrome bridge).

Wire format (contract v1.1): {type, path, note, ts} plus routing extras. Types included `NEW_OUTPUT_READY`, `QUESTION_FOR_YOU`, `NEEDS_INPUT`, `TASK_COMPLETE`, `ACK`, `SPAWN_SESSION`.

Honest ceilings recorded in July (not smoothed)

- Not real-time. File-paced (seconds to minutes).
- Browser legs brittle. Prefer vendor APIs; fall back to DOM.
- More nodes = more drift (fake-DONE / contradictory state). Mitigations named

then: append-only log, single-source registry, adversarial cross-check. Keep topology supervisor + fan-out, not all-to-all.

What COSMOS kept, and what it replaced

BTS-MESH is the **predecessor mesh**. COSMOS is the **OS** that took the same job - multi-AI coordination without Keith as the wire - and put **authority** in one resident Core:

BTS-MESH (July)	COSMOS (ratified 2026-08-23)
File bus + registry as coordination	One signed JSONL ledger as authority; projections rebuild
Orchestrator daemon as conductor	Core = API + scheduler + arbiter + sole ledger writer
Atomic temp+rename payloads	Fenced commit gateway + fencing tokens
Session seed files	Signed SEED . json at session close (OPEN_CONTEXT if missing)
Capability cards	Registry + rails matrix; DOM-first policy
Keith still in some loops	CCr one writer; orch does not hold the COSMOS pen

Continuity of **operation** (July run to August GitHub to live Core) is provenance. Continuity of **code** is successor architecture, not a claim that the July bus is the 2026-08-23 ledger.

What this is not

Not a dump of the BTS tree into a patent. Not keys, Drive folder IDs, or OAuth. Not a claim that GitHub bts-mesh disclosed the method in tracked files (it did not, for CRUCIBLE). Not permission to flip GitHub private from this TUI.

Attach to

P07 (Core as successor authority), P05 (occupancy / multi-AI coordination), P11 (seed / carry-over), and the provenance note Legal wants on the public record. Duplicate into each FILE provisional that needs July operation as support for 112(a) or for inventor-grace timing. Legal copies further excerpts from v:\Ai\BTS_MESH; this TUI does not.